Context Engineering

Context, the context window, memory, and the four core techniques

---

1. What is context?

A large language model (LLM) such as Llama 3.2 **does not remember anything between requests**. Each time an application calls it, the model starts fresh. The only thing it knows about the current situation is the text sent in that request: the **context**.

Think of the model as a capable employee who forgets everything after each reply. Every time a customer writes in, you hand them a sheet of paper with everything they need: the rules, who the customer is, what was said before, and the question. That sheet of paper is the context.

Chat models receive the context as a list of messages, each with a **role**:

```
[system]     You are GadgetHub's support assistant. Returns: 30 days,
             or 60 for Plus members. Headphones must be unopened.
[user]       Hi, I'm a Plus member and I bought headphones.
[assistant]  Thanks! How can I help?
[user]       I opened the box. Can I return them?
```

> **Definition**
>
> **Context engineering** is deciding what information goes into the model's context on every request, and how it is organised, so the model can answer well. Most wrong answers are context problems: something was missing, irrelevant, or out of date.

2. The context window

Models read text as **tokens**, small chunks of about three-quarters of a word (roughly four characters in English). The **context window** is the maximum number of tokens a model can read in one request. Llama 3.2 supports up to 128,000 tokens, although tools such as Ollama often run it with a smaller window by default (set with `num_ctx`).

The window holds **everything** for one request: instructions, history, documents and the question. Treat it as a budget:

- **Overflow**: anything beyond the limit is cut off, usually without warning, so the model never sees it.
- **Cost and speed**: every token is processed on every request, so longer contexts are slower and, with paid APIs, more expensive.
- **Distraction**: important facts buried in irrelevant text are more likely to be missed or mixed up.

3. Memory and storage

Because the model itself is stateless, any "memory" an application has is built **around** the model and fed into the context when needed.

Type	Where it lives	How long it lasts	GadgetHub example
Model knowledge	Inside the model's weights, learned in training	Fixed until the model is retrained	What a refund or warranty is in general
Short-term memory	The context window: the current conversation history	One conversation; lost when the chat ends or the window overflows	"I opened the box" said two messages ago
Long-term memory	External storage: a file or database	Across sessions, until deleted	"Customer is a Plus member", saved last week
Knowledge store	External documents, often in a vector database	Until the documents change	The returns, shipping and warranty policies

How conversation history works

Chat apps seem to remember the conversation, but the model doesn't. The application keeps the list of messages and **sends the whole list again on every turn**. That's why history grows with every message, why long chats can overflow the window, and why starting a new chat means the model knows nothing until the application adds information back.

How long-term memory works

Long-term memory is ordinary storage (a JSON file, a database table, a vector database) that the application **writes** to when something worth keeping comes up, and **reads** from when building a new context. The model never accesses storage directly; it only sees what the application puts into the context.

4. The four core techniques

Context engineering is usually described as four techniques for getting the right information into a limited window:

Technique	Idea	Analogy
Write	Save information outside the context so it can be brought back later	A notebook
Select	Put only what's relevant to the current question into the context	A librarian who brings the right book

Technique	Idea	Analogy
Compress	Shrink what you keep while preserving the key facts	Meeting minutes instead of a transcript
Isolate	Give separate sub-tasks their own small, focused contexts	A team of specialists

Write: save it for later

Store useful facts **outside** the model so they survive beyond one request or conversation, then read them back when needed. This is how long-term memory is built. Example: the customer says "I'm a Plus member"; the app saves it to `memory.json`. A week later, in a new chat, the app loads it into the system message, and the model can answer "Is express shipping free for me?" correctly. AI agents also write **scratchpad** notes (plans, progress) for the same reason.

Select: pick what's needed

For each request, include only the information relevant to it. Example: out of all help articles, only the "Shipping times" article is sent for a shipping question; returns, payments and unrelated notes are left out. Real systems search by meaning using **embeddings** and a **vector database**; retrieving documents and adding them to the prompt is called **RAG** (Retrieval-Augmented Generation). The same idea applies to selecting which memories, tools or examples to include.

Compress: make it smaller

Reduce the tokens used while keeping the important facts. The two common approaches for chat history are **trimming** (keep only the last few messages; cheap, but older facts are lost) and **summarising** (the model replaces older messages with a short summary and recent messages are kept as they are). Example: an eight-message chat becomes one summary sentence, "Plus member, bought headphones", plus the last two messages. Large tool results can be summarised the same way.

Isolate: keep jobs separate

Split a task into sub-tasks and give each its own small context, then combine the results. Example: "What payment methods do you accept, and how long is the warranty?" becomes two separate requests, one with only the payment rules and one with only the warranty rules. Multi-agent systems work this way: sub-agents work in their own clean contexts and return only short results, keeping the main context small.